

PYTHON FOR QUANT TRADERS · PART 0

รากฐาน — ก่อนเขียนโค้ดบรรทัดแรก

คณิตศาสตร์ · สถิติ · ตลาดการเงิน · Quant Mindset · Roadmap

ออกแบบสำหรับทุกคน — ไม่ว่าจะจบสายอะไร

วิสัยทัศน์

โลกเปลี่ยนแล้ว — ไม่ต้องจบสายการเงินหรือ CS ก็เขียนโค้ดวิเคราะห์ตลาดได้
หมอบที่อ่านหนังสือเล่มนี้จบจะเห็นว่า p -value ในงานวิจัยยากับ p -value ใน regression
ของราคาหุ้น คือสิ่งเดียวกัน หนายที่อ่านจบจะเห็นว่า LP constraint $x_1 + x_2 \leq 1$ คือ
"เงื่อนไขสัญญาณ" ในภาษาคณิตศาสตร์

Part 0 ปูพื้น 5 เรื่องที่จำเป็น — ทั้งหมดเชื่อมตรงกับโค้ดที่จะเขียนใน **Part I** ถึง **VI**

แผนที่หนังสือ — 35 บท 7 ส่วน

Part 0

รากฐาน
บท 1–5

◀ อยู่ที่นี่

Part I

Python Basics
บท 6–10

Part II

Math Tools
บท 11–17

Part III

OOP
บท 18–21

Part IV

Backtesting
บท 22–25

Part V

AI-Assisted
บท 26–30

Part VI

Live Trading
บท 31–35

บทที่ 1: คณิตศาสตร์ที่ต้องรู้จริงๆ

แก่นของบทนี้

คณิตศาสตร์ทั้งหมดใน Quant สรุปได้เป็น 4 ความคิด: ฟังก์ชัน (input→output), ผลรวม Σ (บวกหลายค่า), เวกเตอร์ (รายการเลข), เมทริกซ์ (ตารางเลข) — ทั้งหมดแปลเป็น Python ได้ตรงตัว

1.1 ฟังก์ชัน — เครื่องแปลงค่า

ฟังก์ชันคือกฎที่บอกว่า "ใส่อะไรเข้าไป ได้อะไรออกมา" เหมือนสูตรอาหาร

$$f(x) = 2x + 3$$

อ่านว่า: "ฟังก์ชัน f ของ x เท่ากับ สองเท่าของ x บวกสาม"

ตัวอย่าง: $f(5) = 2(5) + 3 = 13$ | $f(10) = 2(10) + 3 = 23$

Running Example: ฟังก์ชันใน Quant

ผลตอบแทนรายวัน (Return) เป็นฟังก์ชันของราคา:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

อ่านว่า: "r ณ เวลา t เท่ากับ ราคาวันนี้ ลบ ราคาเมื่อวาน หารด้วย ราคาเมื่อวาน"

ตัวอย่าง: $P_{t-1} = 100, P_t = 103 \rightarrow r_t = \frac{103-100}{100} = 3\%$

ใน Python: `r = (price_today - price_yesterday) / price_yesterday` — เราจะเขียนสิ่งนี้ใน Part I

ฟังก์ชันหลายตัวแปร

ฟังก์ชันรับ input ได้มากกว่า 1 ตัว เช่น ความดันโลหิตเป็นฟังก์ชันของหลายปัจจัย:

$$BP = \alpha + \beta_1 \cdot \text{น้ำหนัก} + \beta_2 \cdot \text{อายุ} + \beta_3 \cdot \text{ออกกำลังกาย}$$

ใน Quant เราทำสิ่งเดียวกัน: ผลตอบแทนหุ้น = ฟังก์ชันของ (ตลาดรวม, อุตสาหกรรม, ข่าว, ...)

1.2 ผลรวม Σ – บวกหลายค่าพร้อมกัน

$$S = \sum_{i=1}^n x_i = x_1 + x_2 + x_3 + \cdots + x_n$$

อ่านว่า: "ซิกมาของ x_i ตั้งแต่ $i=1$ ถึง n " = "บวก x ทุกตัวเข้าด้วยกัน"

ตัวอย่าง: ผลกำไรรายสัปดาห์

กำไร 5 วัน: $x = [+200, -150, +300, -80, +120]$ บาท

$$\sum_{i=1}^5 x_i = 200 + (-150) + 300 + (-80) + 120 = +390 \text{ บาท}$$

ใน Python: `sum([200, -150, 300, -80, 120])` คืนค่า 390

► แบบฝึกหัด: คำนวณกำไรรวม

ผลตอบแทนรายวัน 4 วัน: $[+1\%, -0.5\%, +2\%, -0.3\%]$

$$\sum_{i=1}^4 r_i = 1 + (-0.5) + 2 + (-0.3) = +2.2\%$$

กำไรรวม (แบบ simple) = 2.2% ของเงินทุนเริ่มต้น

1.3 เวกเตอร์ — รายการตัวเลขที่เรียงลำดับ

เวกเตอร์คือ รายการตัวเลข ไม่มีอะไรน่ากลัวกว่านั้น ใน Python คือ list หรือ numpy array

$$\mathbf{P} = [100, 105, 98, 110, 107]$$

แปลว่า: ราคาหุ้น 5 วัน (เวกเตอร์ขนาด 5 มิติ)

คอลัมน์ Python ในตารางนี้แค่ให้เห็นภาพล่วงหน้า — ยังไม่ต้องเข้าใจ เราจะเรียนเขียนจริงใน Part I

Operation พื้นฐานบนเวกเตอร์

Operation	ความหมาย	Python
$\mathbf{x} + \mathbf{y}$	บวกทีละ element	<code>x + y (numpy)</code>
$c \cdot \mathbf{x}$	คูณทุก element ด้วย c	<code>c * x</code>
$\mathbf{x} \cdot \mathbf{y}$	dot product: $\sum x_i y_i$	<code>np.dot(x, y)</code>
$\ \mathbf{x}\ $	ความยาวเวกเตอร์: $\sqrt{\sum x_i^2}$	<code>np.linalg.norm(x)</code>

1.4 เมทริกซ์ — ตาราง Excel ที่มีตัวเลขทุกช่อง

เมทริกซ์คือตาราง $m \times n$ (m แถว, n คอลัมน์) ใน Python คือ pandas DataFrame หรือ numpy 2D array

$$\mathbf{A} = \begin{pmatrix} 100 & 200 & 150 \\ 105 & 195 & 148 \\ 98 & 210 & 155 \end{pmatrix}$$

แถว = วัน | คอลัมน์ = หุ้น (AAPL, MSFT, NVDA)

$A[0][0] = 100$ หมายถึง AAPL วันที่ 1 ราคา 100

Matrix ที่สำคัญที่สุดใน Quant: Covariance Matrix

เมทริกซ์ความแปรปรวนร่วมบอกว่าแต่ละคู่หุ้นเดินสัมพันธ์กันแค่ไหน:

$$\Sigma = \begin{pmatrix} \sigma_A^2 & \sigma_{AB} \\ \sigma_{AB} & \sigma_B^2 \end{pmatrix}$$

ตัวเลขในแนวทแยง = ความแปรปรวนของแต่ละหุ้น | นอกแนวทแยง = ความแปรปรวนร่วมของคู่หุ้น

ใช้ใน Part II สำหรับ portfolio optimization

สรุปความเชื่อมโยงกับ Python

คณิตศาสตร์	ใน Python	เริ่มเรียนที่
ฟังก์ชัน $f(x)$	<code>def f(x): return ...</code>	Part I บทที่ 7
ผลรวม Σ	<code>sum() / np.sum()</code>	Part I บทที่ 9
เวกเตอร์	<code>list / np.array</code>	Part I บทที่ 9
เมทริกซ์	<code>pd.DataFrame</code>	Part I บทที่ 9

บทที่ 2: สถิติสำหรับมนุษย์

แก่นของบทนี้

สถิติคือการ "สรุปข้อมูลจำนวนมากให้เป็นตัวเลขไม่กี่ตัว" 5 concept ที่ต้องรู้: ค่าเฉลี่ย, ส่วนเบี่ยงเบน, การกระจายปกติ, correlation, และ regression — ทั้งหมด Python คำนวณได้ใน 1-2 บรรทัด แต่ต้องรู้ ความหมาย ก่อนจึงจะตีความผลลัพธ์ได้ถูก

2.1 ค่าเฉลี่ยและส่วนเบี่ยงเบน — อยู่ตรงไหน กระจายแค่ไหน?

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n} \quad \sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$$

อ่านว่า: " $\bar{x}$ (x-bar) = ค่าเฉลี่ย" | " σ (sigma) = ส่วนเบี่ยงเบนมาตรฐาน = ค่าเฉลี่ยของ ระยะห่าง จากค่าเฉลี่ย"

Running Example: ผลตอบแทนรายวัน

ผลตอบแทน 5 วัน: $r = [+1\%, -0.5\%, +2\%, -1\%, +0.5\%]$

$$\bar{r} = \frac{1 + (-0.5) + 2 + (-1) + 0.5}{5} = \frac{2}{5} = 0.4\%/\text{วัน}$$

$$\sigma \approx 1.08\%/\text{วัน}$$

แปลว่า: วันปกติผลตอบแทนอยู่ระหว่างประมาณ -0.68% ถึง $+1.48\%$ ($\pm 1\sigma$ จาก $\bar{r}$)

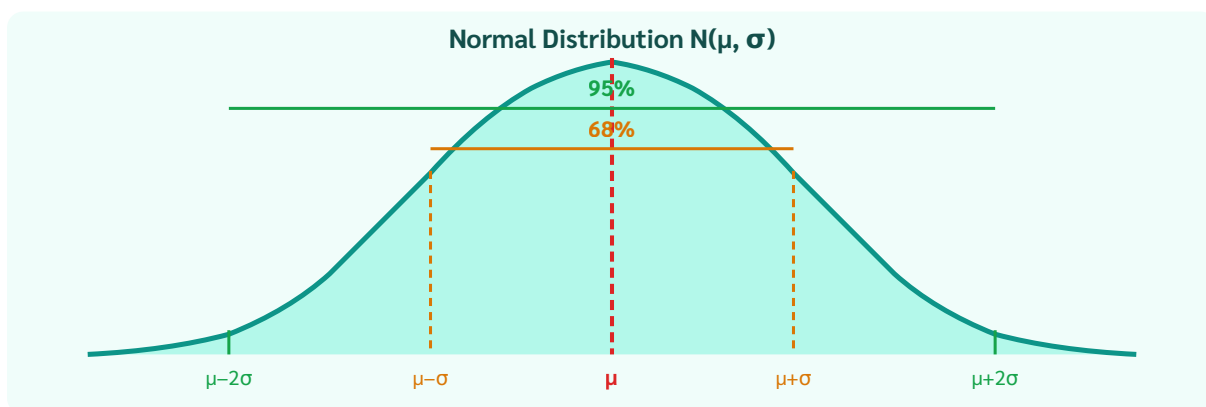
ใน Python: `np.mean(r)` และ `np.std(r)`

ความหมายของ σ ในการเทรด

สินทรัพย์	σ รายวัน	แปลว่า
US Treasury Bond	~0.05%	นิ่งมาก, ผลตอบแทนต่ำ
หุ้นขนาดใหญ่ (S&P500)	~1%	ปกติสำหรับหุ้น
BTC	~3-5%	ผันผวนสูง, โอกาส/ความเสี่ยงมาก
Altcoin ขนาดเล็ก	~10%+	เก็งกำไรสูงมาก

2.2 Normal Distribution — รูประฆัง

ข้อมูลหลายอย่างในธรรมชาติกระจายเป็น "รูประฆัง" (Bell Curve) รอบค่าเฉลี่ย μ ด้วยความกว้าง σ



กฎ 68-95-99.7: ข้อมูล 68% อยู่ใน $\mu \pm \sigma$, 95% อยู่ใน $\mu \pm 2\sigma$, 99.7% อยู่ใน $\mu \pm 3\sigma$

ตัวอย่าง: กฎ 68-95-99.7 กับหุ้น

BTC มีผลตอบแทนรายวัน $\mu = 0\%$, $\sigma = 3\%$

- วันส่วนใหญ่ (68%) BTC เคลื่อนที่ระหว่าง -3% ถึง $+3\%$
- วันที่ตก $>6\%$ (เกิน 2σ) เกิดได้ประมาณ 5% ของวันทั้งหมด = ประมาณ 12-13 วัน/ปี
- วันที่ตก $>9\%$ (เกิน 3σ) เกิดได้ประมาณ 0.3% = ประมาณ 1 วัน/ปี

2.3 Correlation — สองตัวแปรเดินไปด้วยกันไหม?

$$r_{XY} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}} \in [-1, +1]$$

$r = +1$: เดินทิศเดียวกันสมบูรณ์ | $r = 0$: ไม่สัมพันธ์ | $r = -1$: เดินสวนทางสมบูรณ์

Correlation $\neq$ Causation (ความสัมพันธ์ $\neq$ เป็นสาเหตุ)

ไอศกรีมขาย correlate กับอัตราจมน้ำ ($r \approx +0.9$) — แต่ไม่ใช่เพราะกินไอศกรีมแล้วจมน้ำ แต่เพราะ "อากาศร้อน" เป็นตัวแปรที่ซ่อนอยู่ (confounding variable)

ใน Quant: สองหุ้นอาจ correlate กันโดย ไม่ cointegrate กัน — ต่างกันมาก เราจะเรียนใน Part II

2.4 p-value — มั่นใจแค่ไหนว่าไม่ใช่ความบังเอิญ?

p-value ตอบคำถามว่า: "ถ้าสมมติว่าไม่มีผลจริง (null hypothesis) โอกาสที่จะเห็นข้อมูลแบบนี้โดยบังเอิญคือเท่าไร?"

Running Example: ทดสอบกลยุทธ์

กลยุทธ์ทำกำไรเฉลี่ย $+0.1\%$ /วัน ใน 252 วัน (1 ปี)

p-value = 0.03 หมายความว่า:

- "ถ้ากลยุทธ์ไม่มีประสิทธิภาพจริง (random) โอกาสที่จะเห็นผลดีขนาดนี้โดยบังเอิญ = 3%"
- ต่ำกว่า 5% = **มีนัยสำคัญทางสถิติ** (statistically significant)

ใน Python ใช้ library ชื่อ statsmodels (จะเขียนโค้ดนี้จริงใน Part II บทที่ 13)

2.5 Regression — หาเส้นที่พอดีที่สุด

สูตรหลัก — OLS REGRESSION

$$y = \alpha + \beta x + \varepsilon$$

α (alpha) = intercept = ค่า y เมื่อ $x = 0$

β (beta) = slope = ถ้า x เพิ่ม 1 หน่วย y เปลี่ยนไป β หน่วย

ε (epsilon) = error = ส่วนที่ model ทำนายไม่ได้

OLS (Ordinary Least Squares) หาค่า α, β ที่ทำให้ $\sum \varepsilon_i^2$ น้อยที่สุด — เรียนลึกใน Part II บทที่ 13

ตัวอย่าง: Hedge Ratio ระหว่างสองหุ้น

เราต้องการหาว่า AAPL เคลื่อนที่ตาม MSFT แค่ไหน:

$$r_{\text{AAPL}} = \alpha + \beta \cdot r_{\text{MSFT}} + \varepsilon$$

ผล OLS: $\beta = 0.85$ หมายความว่า

→ ทุกครั้งที่ MSFT ขึ้น 1% AAPL มักขึ้น 0.85%

→ ถ้าต้องการ hedge AAPL \$100K ด้วย MSFT → ต้อง short MSFT \$85K

```
# Python (Part II)
import statsmodels.api as sm
X = sm.add_constant(r_msft)
model = sm.OLS(r_aapl, X).fit()
beta = model.params['r_msft']    # = 0.85
```

► แบบฝึกหัด: อ่านผล regression

สมมติ regression ผลตอบแทน BTC บน ETH ได้: $\beta = 1.12, \alpha = 0.001, R^2 = 0.78$

แปลว่า:

- ETH ขึ้น 1% → BTC มักขึ้น 1.12% (BTC volatile กว่า ETH เล็กน้อย)
- $R^2 = 0.78 = 78\%$ ของความผันผวน BTC อธิบายได้ด้วย ETH
- 22% ที่เหลือ (ε) คือ "idiosyncratic move" ที่ไม่สัมพันธ์ — นี่คือ spread ที่จะ trade

บทที่ 3: ตลาดการเงิน 101

แก่นของบทนี้

ไม่ต้องรู้จักหุ้นทุกตัว แค่รู้ 4 concept: **สินทรัพย์** (มีอะไรซื้อขายได้บ้าง), **Return** (วัดผลกำไร/ขาดทุนอย่างไร), **Risk = Volatility** (วัดความไม่แน่นอน), **Arbitrage** (หลักการหากำไรไร้ความเสี่ยง) — 4 concept นี้รองรับโค้ดที่จะเขียนตลอดเล่ม

3.1 ประเภทสินทรัพย์

ประเภท	ความหมายง่ายๆ	ตัวอย่าง
หุ้น (Stocks)	ส่วนแบ่งความเป็นเจ้าของบริษัท — กำไรขาดทุนตามบริษัท	AAPL, PTT, ADVANC
พันธบัตร (Bonds)	กู้ยืมเงิน ได้ดอกเบี้ยแน่นอน — ความเสี่ยงต่ำกว่าหุ้น	US Treasury, พันธบัตรรัฐบาลไทย
FX / Crypto	แลกเปลี่ยนระหว่างสกุลเงิน — ราคาขึ้นลงตามอุปสงค์อุปทาน	USD/THB, BTC/USDT
Derivatives	สัญญาที่ราคาขึ้นกับสินทรัพย์อื่น — leverage สูง, ซับซ้อน	Futures, Options, Perpetuals

3.2 Price vs Return

นักวิเคราะห์ทำงานกับ Return มากกว่า Price เพราะ Return เปรียบเทียบข้ามสินทรัพย์ได้

$$r_t^{\text{simple}} = \frac{P_t - P_{t-1}}{P_{t-1}}$$
$$r_t^{\text{log}} = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Simple vs Log Return — เลือกอันไหน?

	Simple Return	Log Return
สูตร	$(P_t - P_{t-1})/P_{t-1}$	$\ln(P_t/P_{t-1})$
บวกข้ามเวลา	ไม่ได้โดยตรง	ได้ (additive)
กระจายเป็น Normal	ใกล้เคียง	ดีกว่า
ใช้เมื่อ	ผลตอบแทนระยะสั้น (ต่ำกว่า 5%)	วิเคราะห์เชิงสถิติ, cointegration

สำหรับค่าน้อยๆ: $\log \text{return} \approx \text{simple return}$ (ต่างกันน้อยมาก)

3.3 Risk = Volatility

ใน Finance **Risk = ความไม่แน่นอน = Standard Deviation ของ Return** เรียกว่า Volatility (σ)

ตัวอย่าง: Annualized Volatility

BTC มี $\sigma_{\text{daily}} = 3\%$ ต่อวัน

แปลงเป็นรายปี: $\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{T}$ โดย T = จำนวนวันซื้อขายต่อปี

$$\sigma_{\text{annual}} = 3\% \times \sqrt{252} \approx 47.6\%/\text{ปี}$$

แปลว่า: ใน 1 ปี ราคา BTC มักเคลื่อนที่ไปมา $\pm 47.6\%$ จากค่าเฉลี่ย

หมายเหตุ: หุ่นไทย/สหรัฐใช้ $\sqrt{252}$ (252 วันทำการ) — แต่ **Crypto** เปิด 24/7 ใช้ $\sqrt{365}$

BTC: $3\% \times \sqrt{365} \approx 57.3\%/\text{ปี}$ — ตัวเลขต่างกันไม่ใช่ผิด แต่ต้องใช้ให้สม่ำเสมอ

3.4 Arbitrage — ซื้อถูก ขายแพง พร้อมกัน

Arbitrage คือการทำกำไรโดยไม่มีความเสี่ยงสุทธิ จากราคาที่ไม่สมดุลกันระหว่างตลาด



Arb หายเองเมื่อราคาสองตลาดบิเข้าหากัน (convergence)

ทำไม Pure Arbitrage ถึงหายากขึ้นเรื่อยๆ?

เมื่อมีคนเห็น Arb → รีบซื้อตลาด A + ขายตลาด B → ราคา A สูงขึ้น, ราคา B ลง → ส่วนต่างหด → หายภายใน ms-วินาที

โอกาสที่เหลือต้องการ: **ความเร็ว** (automated bot), **ข้อมูลที่คนอื่นไม่มี**, หรือ **capital ขนาดใหญ่** สำหรับโอกาสที่ margin น้อย

3.5 Portfolio และ Diversification

$$R_p = \sum_{i=1}^n w_i R_i \quad \sum_{i=1}^n w_i = 1$$

อ่านว่า: "ผลตอบแทน portfolio เท่ากับ ผลรวมถ่วงน้ำหนักของผลตอบแทนแต่ละสินทรัพย์"

Diversification: กระจายเพื่อลด Risk โดยไม่ลด Return

ถ้าหุ้น A และ B มี $r = -1$ (สวนทางสมบูรณ์) → ถือ 50%/50% → portfolio ไม่ขึ้นไม่ลง

ถ้า $r = 0$ (ไม่สัมพันธ์) → $\sigma_p = \sigma / \sqrt{2}$ = ลด risk ได้ 30% โดยไม่ลด return

กุญแจ: หา assets ที่ correlation ต่ำ เพื่อกระจาย risk ได้จริง

บทที่ 4: คิดแบบ Quant

แก่นของบทนี้

ทักษะ coding และ math เป็นแค่เครื่องมือ สิ่งที่แยก Quant ออกจากคนทั่วไปคือ **วิธีคิด** — ตัดสินใจด้วยตัวเลข ไม่ใช่ความรู้สึก, รู้ว่าโมเดลมีขีดจำกัด, คิดเรื่อง expected value ก่อนเสมอ

4.1 ทุกอย่างคือ Data

สิ่งที่เห็น	Quant มองว่า	ใช้ทำอะไร
ราคาหุ้น	Time series ของ returns	หา pattern, signal
ข่าว	Sentiment score [-1, +1]	NLP model ทำนายทิศ
ปริมาณซื้อขาย	Volume series	ยืนยัน signal, หา liquidity
Fear ตลาด	VIX index	ปรับ position size
คำพูด Fed	Hawkish/Dovish score	ปรับ model สำหรับ macro regime

Running Example: หมอกับ Quant คิดคล้ายกัน

หมอไม่ "รู้สึก" ว่าคนไข้เป็นอย่างไร — วัด: BP, HR, SpO₂, GCS, Temp

Quant ไม่ "รู้สึก" ว่าตลาดจะขึ้นหรือลง — ดู data: price, volume, σ , correlation, macro

ทั้งคู่ ตัดสินใจจาก signal ที่วัดได้ ไม่ใช่ intuition — แต่ทั้งคู่ก็รู้ว่า signal ไม่สมบูรณ์เสมอ

4.2 Model = แผนที่ ≠ ดินแดน

"All models are wrong, but some are useful" — George E.P. Box

แผนที่กรุงเทพฯ ไม่ใช่กรุงเทพฯ จริง — ไม่มีต้นไม้ ไม่มีเสียง ไม่มีกลิ่น
แต่แผนที่ช่วยนำทางได้ — และนั่นคือสิ่งที่เราต้องการ

โมเดล Quant เช่นกัน: ไม่ได้อธิบายตลาดครบทุกมิติ แต่ช่วยตัดสินใจได้ดีกว่าไม่มีโมเดล
อันตรายคือการลืมนึกว่าโมเดลคือแผนที่ แล้วเชื่อว่ามันคือความจริงสมบูรณ์

4.3 Expected Value — ตัดสินใจด้วยค่าคาดหวัง

สูตรหลัก

$$E[X] = \sum_i p_i \cdot x_i$$

p_i = ความน่าจะเป็นที่ผลลัพธ์ i จะเกิด (ทุก p_i รวมกัน = 1)

x_i = มูลค่าของผลลัพธ์ i (บวก = กำไร, ลบ = ขาดทุน)

Quant คุณที่ $E[X]$ ก่อนเสมอ ไม่ใช่แค่ "น่าจะชนะ"

ตัวอย่าง: เปรียบเทียบสองกลยุทธ์ด้วย Expected Value

กลยุทธ์ A

ชนะ 70% ได้ +฿100 | แพ้ 30% เสีย -฿300

ชนะ 70%
+฿100 → +฿70

แพ้ 30%
-฿300 → -฿90

$E[A] = +70 - 90 = -฿20$

กลยุทธ์ B

ชนะ 40% ได้ +฿800 | แพ้ 60% เสีย -฿200

ชนะ 40%
+฿800 → +฿320

แพ้ 60%
-฿200 → -฿120

$E[B] = +320 - 120 = +฿200$ ✓

กลยุทธ์ A ชนะบ่อยกว่า (70%) แต่ $E[X]$ ติดลบ — กลยุทธ์ B แพ้บ่อยกว่า (60%) แต่ $E[X]$ เป็นบวกมาก

4.4 Backtesting — ทดสอบก่อนใช้เงินจริง

Backtesting คือการนำกลยุทธ์ไป "เล่นซ้ำ" กับข้อมูลราคาในอดีตเพื่อดูว่าถ้าใช้ในอดีตจะได้ผลลัพธ์อย่างไร

อันตราย: Overfitting = การเรียน "คำตอบสอบ" ไม่ใช่ "วิชา"

ถ้าปรับ parameter มากพอ กลยุทธ์ใดๆ ก็ดูดีในข้อมูลเก่าได้เสมอ
เหมือนจำคำตอบข้อสอบ — ผ่านข้อสอบเก่าได้ แต่ข้อสอบจริง (live market) ไม่ได้

วิธีป้องกัน: ทดสอบกับข้อมูลที่โมเดลไม่เคยเห็น (out-of-sample) — เราจะเรียนใน Part IV

► แบบฝึกหัด: คิดแบบ Quant

กลยุทธ์นี้ดีไหม? "ทำกำไร 8 ใน 10 ครั้ง"

ข้อมูลไม่พอ ต้องถามเพิ่ม:

1. ครั้งที่แพ้ 2 ครั้ง ขาดทุนเท่าไร? → ถ้าแพ้ครั้งละ $-5,000$ บาท และชนะครั้งละ $+100$ บาท → $E[X]$ ติดลบ
2. ทดสอบกับข้อมูล out-of-sample ไหม? → ถ้าทดสอบแค่ in-sample อาจ overfit
3. ค่า transaction cost รวมอยู่ด้วยไหม? → ถ้าไม่รวม ผลจริงอาจแย่มาก

บทที่ 5: Roadmap & วิธีอ่านหนังสือเล่มนี้

แก่นของบทนี้

หนังสือเล่มนี้ออกแบบให้ **อ่านตามลำดับ** — แต่ละ Part สร้างบนของเดิม ถ้าติดในบท X ให้อ่านต่อไปก่อน เพราะบางครั้ง context ข้างหน้าช่วยให้เข้าใจย้อนกลับมาได้ดีกว่า

5.1 AI ช่วยได้ที่ไหนบ้าง?

หนังสือเล่มนี้ไม่ได้สอนให้ "จำ syntax" — สอนให้ **รู้ว่าการทำอะไร และตรวจสอบ AI ได้**

AI ทำได้ดี	คุณต้องรู้เอง
เขียน boilerplate / template code	ว่า algorithm ที่ต้องการคืออะไร
แปลง logic เป็น Python syntax	ว่า logic นั้นถูกต้องทาง math ไหม
Debug error message	ว่า root cause จริงคืออะไร
เขียน OLS / optimization ให้	ว่า β หรือ output ที่ได้มีความหมายอะไร
หา library และ function ที่เหมาะสม	ว่าผลลัพธ์สมเหตุสมผลไหม

Running Example: Prompt ที่ดีสำหรับ Quant

```
# Prompt ที่ไม่ดี:
"เขียน trading bot ให้หน่อย"

# Prompt ที่ดี:
"เขียน Python function ที่รับ DataFrame ของราคา 2 assets
คืนค่า hedge ratio beta จาก OLS regression (log prices)
และ z-score ของ spread: log(P1) - beta * log(P2)
ใช้ statsmodels.OLS, normalize z-score ด้วย rolling window 60 วัน"
```

Prompt ที่ดีต้องระบุ: input, output, method, library — เราจะเรียนใน Part V

5.2 Learning Path ตามอาชีพ

พื้นฐานที่มี	Part ที่ต้องใส่ใจเป็นพิเศษ	ข้อได้เปรียบ
หมอ / นักวิทยาศาสตร์	Part II (statistics ลึก)	p-value, hypothesis testing คำนวณ
วิศวกร / IT	Part III, VI (OOP, async)	programming concepts คำนวณ
นักการเงิน / Trader	Part I (Python basics)	ตลาด, risk management คำนวณ
ทนาย / นักบัญชี	Part II (LP constraints)	คิด constraint, rule-based ได้ดี
ไม่มีพื้นฐานใดเลย	Part 0 อ่านซ้ำๆ ทำแบบฝึกหัดทุกข้อ	ไม่มี bad habit จากสาขาอื่น

5.3 Environment ที่ต้องติดตั้งก่อน Part I

- 1. Python 3.10+ — ภาษาหลัก ดาวน์โหลดจาก python.org
 - 2. VS Code — โปรแกรมเขียนโค้ด (ฟรี) + Python extension
 - 3. Conda หรือ pip — จัดการ library
 - 4. AI Assistant (Claude, ChatGPT, Gemini) — คู่หูตลอดการเรียนรู้
- บทที่ 6 สอนขั้นตอนการติดตั้งทีละขั้นตอนพร้อม screenshot — ไม่ต้องทำตอนนี้

5.4 Checklist ก่อนเริ่ม Part I

▶ ตรวจสอบความพร้อม — คลิกเพื่อดู

ถ้าตอบ "ใช่" ได้ทุกข้อ พร้อมแล้ว:

- ฟังก์ชัน $f(x) = 2x + 3$ แปลว่า "ใส่ x เข้าไป คูณ 2 บวก 3 ออกมา" ✓
- $\sum_{i=1}^3 x_i$ เมื่อ $x = [10, 20, 30]$ มีค่าเท่ากับ 60 ✓
- Standard Deviation ใหญ่ = ข้อมูลกระจายมาก = ผันผวนมาก ✓
- Correlation $r = -0.8$ หมายถึง X ขึ้นเมื่อ Y ลง (สวนทางกัน) ✓
- Return รายวัน 3% = $(P_t - P_{t-1}) / P_{t-1} = 0.03$ ✓
- Arbitrage ต้องซื้อและขายพร้อมกัน ไม่ใช่ซื้อก่อนแล้วรอขาย ✓
- $E[X] = 0.7 \times (+100) + 0.3 \times (-300) = -20$ ✓

ถ้ายังไม่แน่ใจบางข้อ → กลับไปอ่านบทนั้นซ้ำได้เลย ไม่ต้องรีบ

จบ Part 0 — คุณพร้อมแล้ว

ตอนนี้คุณมีเครื่องมือทางความคิดครบ: **Math** (ฟังก์ชัน, Σ , เวกเตอร์, เมทริกซ์) + **สถิติ** (mean, σ , correlation, regression) + **Finance** (assets, return, risk, arb) + **Quant mindset** (EV, model = map, backtest)

ใน Part I เราจะเริ่มแปลง concept เหล่านี้เป็น Python code ที่รันได้จริง — บรรทัดแรกของคุณรอคุณอยู่ในบทที่ 6